Git (zsh) Use Cases Cheat Sheet — Local + Remote (HTTPS) Generated 2025-12-30

This PDF is a practical, command-focused reference for common Git workflows from a **zsh** shell. Examples assume: You are inside a project directory Your remote host is GitHub (HTTPS) Your GitHub token is stored as GITHUB_TOKEN (recommended)

0) Quick prerequisites (recommended)

Configure a credential helper on macOS so you don't re-enter the token every push:

```
git config --global --replace-all credential.helper osxkeychain
```

Optional: store a token from an environment variable into Git's credential store (no interactive prompts):

```
printf
"protocol=https\nhost=github.com\nusername=<YOUR_GITHUB_USER>\npassword=%s\n\n"
"$GITHUB_TOKEN" | git credential approve
```

1) Create a local repo

Initialize a new repository in the current folder:

```
git init
```

Create an initial commit (typical sequence):

```
git add .
```

```
git commit -m "Initial commit"
```

2) Create a new branch (and switch to it)

Modern Git:

```
git switch -c feature/my-work
```

Classic equivalent:

```
git checkout -b feature/my-work
```

3) Check which branch is active

```
git branch
```

Or more detailed status (shows branch + staged/unstaged changes):

```
git status
```

Or show current branch name only:

```
git rev-parse --abbrev-ref HEAD
```

4) Create a .gitignore + best practices

Create a .gitignore file:

```
touch .gitignore
```

Common best-practice entries (tune for your project):

```
# Python __pycache__/ *.py[cod] *.so *.egg-info/ dist/ build/ .venv/ venv/ # Jupyter
.ipynb_checkpoints/ # OS / editor .DS_Store .idea/ .vscode/ # Logs / caches *.log
.cache/ .mypy_cache/ .pytest_cache/ # Secrets (never commit) .env .env.*
```

Best practices: Ignore **generated** artifacts (build outputs, caches) and **machine-specific** files. Do **not** ignore source files required to build/run (keep the repo reproducible). Never commit secrets: API keys, tokens, .env files, credentials. If you accidentally committed a secret, rotate it immediately (changing .gitignore is not enough).

5) Add files to the local repo (stage changes)

Stage specific files:

```
git add path/to/file1 path/to/file2
```

Stage everything under the current directory:

```
git add .
```

Interactively stage parts of files (very handy):

```
git add -p
```

6) Check which files changed but still need git add

Show unstaged/staged changes summary:

```
git status
```

Show file-level changes (unstaged):

```
git diff --name-only
```

Show what is staged (ready to commit):

```
git diff --cached --name-only
```

7) Commit files to the local repo

Commit staged changes:

```
git commit -m "Describe the change"
```

Amend last commit (message or staged content) — use carefully if already pushed:

```
git commit --amend
```

8) Add an annotated tag to the local repo

Annotated tags are recommended for releases (they include metadata + message):

```
git tag -a v0.1.0 -m "Release v0.1.0"
```

Tag a specific commit (optional):

```
git tag -a v0.1.0 <commit-sha> -m "Release v0.1.0"
```

9) List all tags in the local repo

```
git tag
```

Show annotated tag details:

```
git show v0.1.0
```

10) Push commits to the remote repo (HTTPS)

First push for a branch (sets upstream):

```
git push -u origin main
```

Later pushes:

```
git push
```

11) Push an annotated tag to the remote repo

Push one tag:

```
git push origin v0.1.0
```

Push all tags:

```
git push --tags
```

12) Create a remote GitHub repo with curl (no gh)

This uses GitHub's REST API. Store your token in an env var (zsh):

```
export GITHUB_TOKEN="ghp_...yourtoken..."
```

Create a repo named python-setup-instructions under your user account:

```
curl -s -X POST https://api.github.com/user/repos \ -H "Authorization: Bearer $GITHUB_TOKEN" \ -H "Accept: application/vnd.github+json" \ -d '{ "name": "python-setup-instructions", "private": false }'
```

Notes: For fine-grained tokens, ensure the token has permission to create repos and write contents.

13) Make the local repo aware of the remote (so git push works)

Add the remote named origin (HTTPS):

```
git remote add origin https://github.com/<YOUR_GITHUB_USER>/python-setup-instructions.git
```

Verify remotes:

```
git remote -v
```

If you already have an origin and need to change it:

```
git remote set-url origin https://github.com/<YOUR_GITHUB_USER>/python-setup-instructions.git
```

14) Verify from command line that the remote repo exists

Check via GitHub API (HTTP 200 means it exists):

```
curl -s -o /dev/null -w "%{http_code}\n" \ -H "Authorization: Bearer $GITHUB_TOKEN" \ https://api.github.com/repos/<YOUR_GITHUB_USER>/python-setup-instructions
```

Check via Git (lists refs if accessible):

```
git ls-remote https://github.com/<YOUR_GITHUB_USER>/python-setup-instructions.git
```

15) Verify you can push commits and annotated tags

Push your branch:

```
git push -u origin main
```

Create and push an annotated tag:

```
git tag -a v0.1.0 -m "Release v0.1.0"
```

```
git push origin v0.1.0
```

Verify tags on remote:

```
git ls-remote --tags origin
```

16) Handy Git commands (quality-of-life)

View history: git log --oneline --decorate --graph --all **See what changed:** git diff (unstaged), git diff --cached (staged) **Undo unstaged changes (file):** git restore path/to/file **Unstage a file:** git restore --staged path/to/file **Rename a branch locally:** git branch -m old new **Delete a local branch:** git branch -d branchname (or -D force) **Delete a tag locally + remote:** git tag -d v0.1.0 then git push origin :refs/tags/v0.1.0 **Stash work:** git stash, git

stash pop **Show configured remotes:** git remote -v **Show current config entry:** git config --global --get credential.helper

17) Common gotchas (HTTPS + tokens)

When Git prompts for “Password” on GitHub HTTPS, paste your **token** (not your GitHub account password). If pushes fail, clear stale credentials in macOS Keychain: security delete-internet-password -s github.com Prefer annotated tags for releases: git tag -a ... Use .gitignore early (before adding large generated directories) to avoid churn.